

Handwritten Digit Classification Using an Artificial Neural Network (ANN) on the MNIST Dataset

ADA ŞAHİN

B.Sc. Nuclear Engineering, Hacettepe University

August 19, 2026

Abstract

In this research, a simple multi-layer artificial neural network (ANN) is trained and evaluated on the MNIST handwritten digit dataset [1] using the PyTorch framework [2]. The model architecture, training procedure, encountered implementation issues, and final results are described in detail. The trained model achieved a test accuracy of 97.37%.

Contents

1	Introduction	2
2	Dataset and Preprocessing	2
3	Model Architecture	2
4	Training Procedure	3
4.1	Loss Function and Optimizer	3
4.2	Training Loop	4
4.3	Implementation Issues Encountered	4
5	Results	4
6	Discussion and Future Work	5

1 Introduction

Handwritten digit recognition is a classical classification problem widely used in machine learning both as an educational benchmark and in real-world applications such as postal code reading and processing. The main purpose of this work is to demonstrate, in a hands-on manner, the full process of designing, training, and evaluating a basic artificial neural network architecture from scratch.

The approach used in this study is based on the supervised learning algorithm: the model is trained on a dataset in which each image is presented together with its correct label.

The remainder of this report is organized as follows. Section 2 describes the dataset and preprocessing steps, Section 3 presents the model architecture, Section 4 covers the training procedure and issues encountered during implementation, Section 5 reports the results, and Section 6 provides a discussion and directions for future work.

2 Dataset and Preprocessing

The MNIST dataset consists of 60,000 training and 10,000 test grayscale images, each 28×28 pixels in size [1]. Each image represents a digit between 0 and 9.

During preprocessing, pixel values were first scaled to the $[0, 1]$ range and then normalized using $\mu = 0.5$, $\sigma = 0.5$, mapping them to the $[-1, 1]$ range:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \quad (1)$$

The data loading and preprocessing code is shown in Listing 1.

```
1 # data loading and preprocessing
2 def get_data_loaders(batch_size=64):
3     transform = transforms.Compose([
4         transforms.ToTensor(), # convert images to tensor (0-255) range to
5         transforms.Normalize((0.5,), (0.5,)) # normalize the images (-1, 1)
6         range
7     ])
8     # download and load the MNIST dataset
9     train_dataset = torchvision.datasets.MNIST(root='./data', train=True,
10         download=True, transform=transform)
11     test_dataset = torchvision.datasets.MNIST(root='./data', train=False,
12         download=True, transform=transform)
13     # create data loaders for training and testing
14     train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=
15         batch_size, shuffle=True)
16     test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=
17         batch_size, shuffle=False)
18     return train_loader, test_loader
```

Listing 1: Data loading and preprocessing function

3 Model Architecture

The model used in this study is a simple artificial neural network composed of fully connected layers. The input layer consists of a flattened vector of size $28 \times 28 = 784$. And the model contains three linear layers and two ReLU activation functions:

- Input layer: 784 neurons
- Hidden layer 1: 128 neurons (ReLU activation)
- Hidden layer 2: 64 neurons (ReLU activation)
- Output layer: 10 neurons (digit classes 0-9)

The forward propagation step applies the following linear transformation at each layer:

$$y = Wx + b \quad (2)$$

where W denotes the weight matrix and b the bias vector. The ReLU activation function is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (3)$$

and introduces the non-linearity required for the network to learn complex patterns. The model definition is shown in Listing 2.

```

1 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2
3 class ANN(nn.Module):
4     def __init__(self):
5         # fixed: class name in super() must match the class itself (ANN, not
6         #         NeuralNetwork)
7         super(ANN, self).__init__()
8         self.flatten = nn.Flatten()
9         self.fc1 = nn.Linear(28*28, 128) # input layer to hidden layer
10        self.relu = nn.ReLU() # activation function
11        self.fc2 = nn.Linear(128, 64) # hidden layer to hidden layer
12        self.fc3 = nn.Linear(64, 10) # hidden layer to output layer
13
14    def forward(self, x):
15        # fixed: the flatten result must be assigned back to x, otherwise
16        # fc1 receives the un-flattened tensor and shapes won't match
17        x = self.flatten(x)
18        x = self.fc1(x)
19        x = self.relu(x)
20        x = self.fc2(x)
21        x = self.relu(x)
22        x = self.fc3(x)
23        return x

```

Listing 2: ANN model class

4 Training Procedure

4.1 Loss Function and Optimizer

The model was trained using cross-entropy loss, the standard loss function for multi-class classification problems:

$$\mathcal{L} = -\log(p_{\text{true class}}) \quad (4)$$

The Adam optimization algorithm [3] was used for weight updates, with a learning rate of $\eta = 0.001$.

4.2 Training Loop

At each training step, the following operations were performed in order: (i) resetting the gradients from the previous step, (ii) computing predictions via forward propagation, (iii) computing the loss value, (iv) computing gradients via backpropagation, [4] and (v) updating the weights. The model was trained for [10] epochs.

```
1 # loss function and optimizer
2 define_loss_and_optimizer = lambda model: (nn.CrossEntropyLoss(), optim.Adam(
    model.parameters(), lr=0.001))
3
4 # train
5 def train_model(model, train_loader, criterion, optimizer, num_epochs=10):
6
7     # get model to train mode
8     model.train()
9     train_losses = []
10    for epoch in range(num_epochs):
11        total_loss = 0
12        for images, labels in train_loader:
13            images, labels = images.to(device), labels.to(device)
14
15            optimizer.zero_grad() # zero the gradients
16            predictions = model(images) # forward pass
17            loss = criterion(predictions, labels) # compute loss
18            loss.backward() # backward pass
19            optimizer.step() # update weights
20
21        total_loss += loss.item()
22        avg_loss = total_loss / len(train_loader)
23        train_losses.append(avg_loss)
24        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}')
```

Listing 3: Training loop

4.3 Implementation Issues Encountered

Several technical issues were encountered and resolved during implementation:

- The output of the `flatten` operation inside `forward()` was not reassigned to the input variable, which caused a shape mismatch error;
- Training and testing calls were duplicated in different parts of the script, causing the model to be trained for more epochs than intended;
- Normalized images were plotted directly during visualization without rescaling, resulting in poor contrast.

Resolving these issues ensured that the model was trained correctly and consistently.

5 Results

The training and test loss over epochs is shown in Figure 1.

The model achieved a test accuracy of 97.37%.

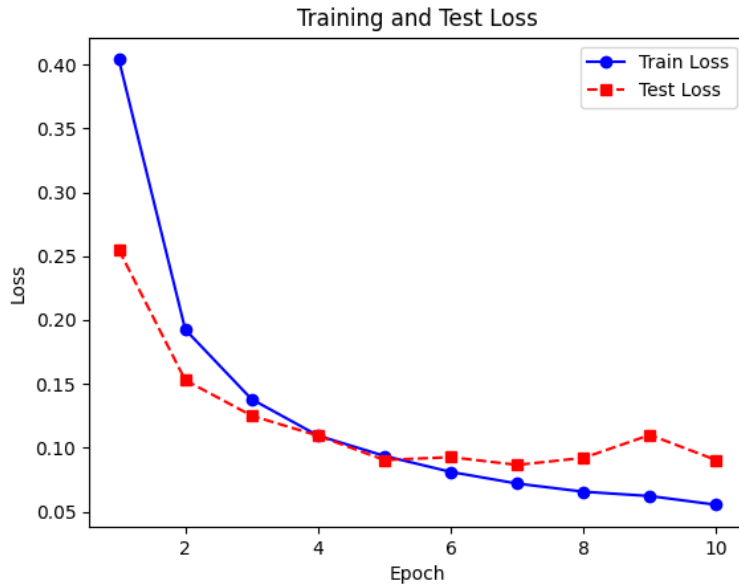


Figure 1: Training and test loss across epochs

Metric	Value
Test accuracy (%)	97.37
Total epochs	10
Learning rate	0.001
Batch size	64

Table 1: Summary of model performance

6 Discussion and Future Work

The results show that even a simple fully connected network can achieve high accuracy on a relatively easy dataset such as MNIST. However, this architecture does not take into account the two-dimensional spatial structure of the images. Future work will focus on a comparative analysis using a Convolutional Neural Network (CNN) architecture. Additionally, techniques such as early stopping and regularization could be explored to detect and mitigate overfitting.

References

- [1] Yann LeCun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.
- [2] Adam Paszke et al. “PyTorch: An imperative style, high-performance deep learning library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [3] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980. 2015.
- [4] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536.